

НАЦИОНАЛЬНЫЙ ИССЛЕДОВАТЕЛЬСКИЙ УНИВЕРСИТЕТ ИТМО
Факультет Программной Инженерии и Компьютерной Техники

Облачные и туманные вычисления
Лабораторная работа № 2
«Миграция в Kubernetes (Переход к Cloud-Native)»

Выполнили
Баянов Равиль Динарович
Группа № Р3434
Преподаватель: Перл Ольга Вячеславовна

г. Санкт-Петербург 2025

Содержание

Содержание	2
Ссылка на репозиторий проекта	3
Трансляция	4
app-deployment.yml.....	4
app-service.yml	5
redis-deployment.yml.....	6
redis-service.yml	7
app-deployment-v2.yml	7
Развертывание.....	10
Rolling Update	13
Вывод.....	14

Ссылка на репозиторий проекта

<https://github.com/RavvChek/ITMO/tree/main/4%20year/%D0%9E%D0%B1%D0%BB%D0%B0%D1%87%D0%BD%D1%8B%D0%B5%20%D0%B8%20%D1%82%D1%83%D0%BC%D0%B0%D0%BD%D0%BD%D1%8B%D0%B5%20%D0%B2%D1%8B%D1%87%D0%B8%D1%81%D0%BB%D0%B5%D0%BD%D0%B8%D1%8F/lab2>

Трансляция

app-deployment.yml

```
apiVersion: apps/v1
kind: Deployment
metadata:
  name: visit-counter
  labels:
    app: visit-counter
spec:
  replicas: 2
  selector:
    matchLabels:
      app: visit-counter
  strategy:
    type: RollingUpdate
    rollingUpdate:
      maxSurge: 1
      maxUnavailable: 0
  template:
    metadata:
      labels:
        app: visit-counter
    spec:
      containers:
        - name: visit-counter
          image: visit-counter:v1
          imagePullPolicy: Never
          ports:
            - containerPort: 5000
          env:
            - name: REDIS_HOST
              value: "redis"
            - name: REDIS_PORT
```

```
    value: "6379"

  resources:
    requests:
      memory: "64Mi"
      cpu: "50m"
    limits:
      memory: "128Mi"
      cpu: "200m"
  readinessProbe:
    httpGet:
      path: /
      port: 5000
    initialDelaySeconds: 5
    periodSeconds: 5
  livenessProbe:
    httpGet:
      path: /
      port: 5000
    initialDelaySeconds: 10
    periodSeconds: 10
```

app-service.yml

```
apiVersion: v1
kind: Service
metadata:
  name: visit-counter
  labels:
    app: visit-counter
spec:
  type: NodePort
  selector:
    app: visit-counter
  ports:
    - port: 5000
```

targetPort: 5000

nodePort: 30080

protocol: TCP

redis-deployment.yml

apiVersion: apps/v1

kind: Deployment

metadata:

name: redis

labels:

app: redis

spec:

replicas: 1

selector:

matchLabels:

app: redis

template:

metadata:

labels:

app: redis

spec:

containers:

- name: redis

image: redis:7-alpine

ports:

- containerPort: 6379

resources:

requests:

memory: "64Mi"

cpu: "50m"

limits:

memory: "128Mi"

cpu: "100m"

redis-service.yml

```
apiVersion: v1
kind: Service
metadata:
  name: redis
labels:
  app: redis
spec:
  type: ClusterIP
  selector:
    app: redis
  ports:
    - port: 6379
      targetPort: 6379
      protocol: TCP
```

app-deployment-v2.yml

```
apiVersion: apps/v1
kind: Deployment
metadata:
  name: visit-counter
labels:
  app: visit-counter
spec:
  replicas: 2
  selector:
    matchLabels:
      app: visit-counter
  strategy:
    type: RollingUpdate
```

rollingUpdate:

maxSurge: 1

maxUnavailable: 0

template:

metadata:

labels:

app: visit-counter

version: v2

spec:

containers:

- name: visit-counter

image: visit-counter:v2

imagePullPolicy: Never

ports:

- containerPort: 5000

env:

- name: REDIS_HOST

value: "redis"

- name: REDIS_PORT

value: "6379"

resources:

requests:

memory: "64Mi"

cpu: "50m"

limits:

memory: "128Mi"

cpu: "200m"

readinessProbe:

httpGet:

path: /

port: 5000

initialDelaySeconds: 5

periodSeconds: 5

livenessProbe:

httpGet:

path: /

port: 5000

initialDelaySeconds: 10

periodSeconds: 10

Развертывание

Запустим minikube

```
(.venv) r.bayanov@macbook-DQMXWK0X05 lab2 % minikube start
🐳 minikube v1.37.0 на Darwin 15.6.1 (arm64)
🔧 Automatically selected the docker driver
🔧 Using Docker Desktop driver with root privileges
👍 Starting "minikube" primary control-plane node in "minikube" cluster
🚚 Pulling base image v0.0.48 ...
📦 Скачивается Kubernetes v1.34.0 ...
  > preloaded-images-k8s-v18-v1...: 332.38 MiB / 332.38 MiB 100.00% 18.64 М
  > gcr.io/k8s-minikube/kicbase...: 450.06 MiB / 450.06 MiB 100.00% 15.20 М
🔥 Creating docker container (CPUs=2, Memory=7788MB) ...
❗ Failing to connect to https://registry.k8s.io/ from inside the minikube container
💡 To pull new external images, you may need to configure a proxy:
https://minikube.sigs.k8s.io/docs/reference/networking/proxy/
🐳 Подготавливается Kubernetes v1.34.0 на Docker 28.4.0 ...
🔗 Configuring bridge CNI (Container Networking Interface) ...
🔍 Компоненты Kubernetes проверяются ...
  ■ Используется образ gcr.io/k8s-minikube/storage-provisioner:v5
🌞 Включенные дополнения: default-storageclass, storage-provisioner
🏠 Готово! kubectl настроен для использования кластера "minikube" и "default" пространства
имён по умолчанию
```

Сборка образа

```
docker build -t visit-counter:v1 .
```

Применим манифесты Kubernetes к кластеру

```
(.venv) r.bayanov@macbook-DQMXWK0X05 lab2 % kubectl apply -f k8s/redis-deployment.yml
deployment.apps/redis created

(.venv) r.bayanov@macbook-DQMXWK0X05 lab2 % kubectl apply -f k8s/redis-service.yml
service/redis created
```

```
(.env) r.bayanov@macbook-DQMXWK0X05 lab2 % kubectl apply -f k8s/app-deployment.yml
deployment.apps/visit-counter created

(.env) r.bayanov@macbook-DQMXWK0X05 lab2 % kubectl apply -f k8s/app-service.yml
service/visit-counter created
```

Проверим внесенные изменения

```
(.env) r.bayanov@macbook-DQMXWK0X05 lab2 % kubectl get pods
```

NAME	READY	STATUS	RESTARTS	AGE
redis-59995c6565-rcpkf	1/1	Running	0	3m22s
visit-counter-6b955c4f99-2b7n7	1/1	Running	0	67s
visit-counter-6b955c4f99-phqqd	1/1	Running	0	67s

```
(.env) r.bayanov@macbook-DQMXWK0X05 lab2 % kubectl get deployments
```

NAME	READY	UP-TO-DATE	AVAILABLE	AGE
redis	1/1	1	1	4m10s
visit-counter	2/2	2	2	115s

```
(.env) r.bayanov@macbook-DQMXWK0X05 lab2 % kubectl get services
```

NAME	TYPE	CLUSTER-IP	EXTERNAL-IP	PORT(S)	AGE
kubernetes	ClusterIP	10.96.0.1	<none>	443/TCP	16m
redis	ClusterIP	10.105.131.155	<none>	6379/TCP	3m10s
visit-counter	NodePort	10.99.218.133	<none>	5000:30080/TCP	2m13s

Запустим приложение

```
(.env) r.bayanov@macbook-DQMXWK0X05 lab2 % minikube service visit-counter
```

NAMESPACE	NAME	TARGET PORT	URL
default	visit-counter	5000	http://192.168.49.2:30080

```
🔧 Starting tunnel for service visit-counter./
```

NAMESPACE	NAME	TARGET PORT	URL
default	visit-counter		http://127.0.0.1:61619

```
🔧 Starting tunnel for service visit-counter.
```



Opening service default/visit-counter in default browser...



Because you are using a Docker driver on darwin, the terminal needs to be open to run it.



Мини-магазин — счётчик посещений

Счётчик: **207**

Запрос обработан контейнером: visit-counter-6b955c4f99-2b7n7

Статус Redis: OK
Хост Redis: redis:6379

Rolling Update

Поменяем версию приложения и соберем новый образ

```
copy app-v2.py app.py  
docker build -t visit-counter:v2 .
```

Применим обновленный deployment

```
kubectl apply -f k8s/app-deployment-v2.yaml
```

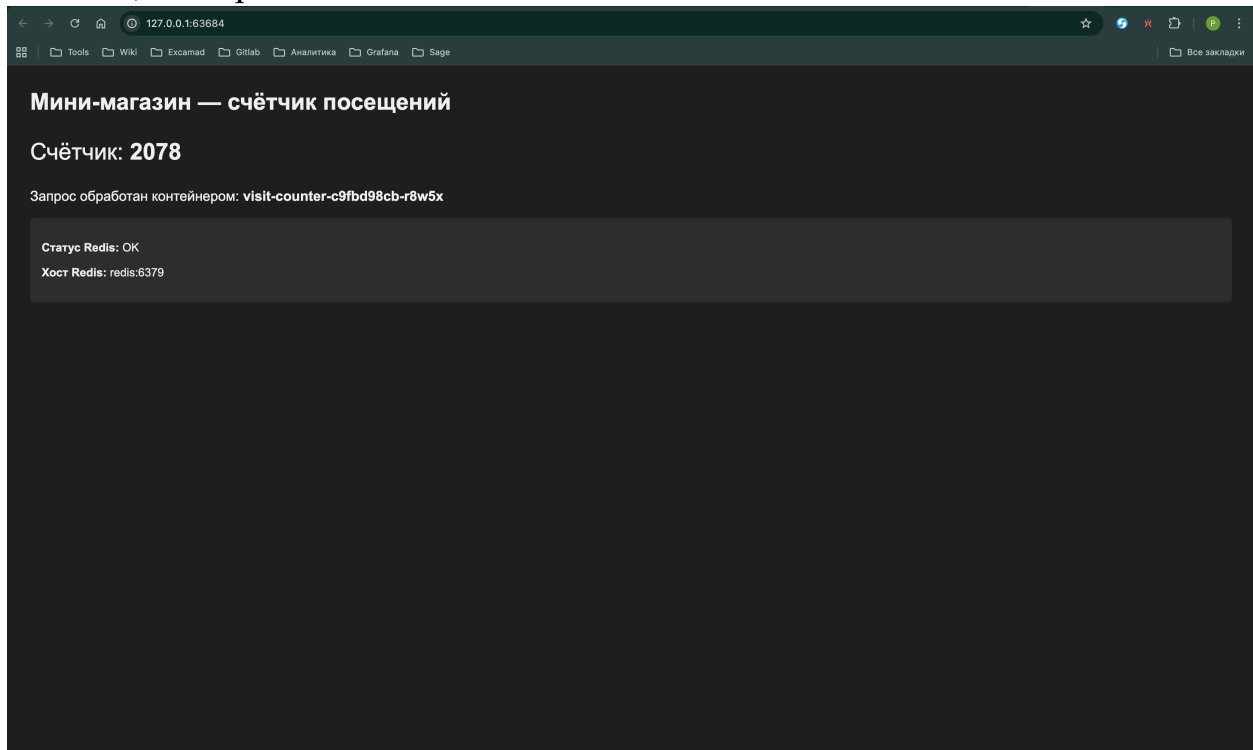
Применим новый образ

```
(.venv) r.bayanov@macbook-DQMXWK0X05 lab2 % kubectl rollout restart deployment/visit-counter  
deployment.apps/visit-counter restarted
```

Видим, что процесс обновления закончился

```
(.venv) r.bayanov@macbook-DQMXWK0X05 lab2 % kubectl rollout status deployment/visit-counter  
deployment "visit-counter" successfully rolled out
```

Увидим, что приложение обновилось без полной остановки



Вывод

В ходе лабораторной работы я перенёс приложение из Docker Compose в Kubernetes, вручную преобразовав конфигурацию в манифесты Deployment и Service и развернув их в локальном кластере Minikube. В процессе работы я изучил жизненный цикл Pod'ов и на практике понял разницу между императивным и декларативным подходами управления инфраструктурой. Также я реализовал rolling update приложения без простоя сервиса, обновив образ контейнера и наблюдая поэтапную замену pod'ов. В результате я получил практический опыт работы с базовыми примитивами Kubernetes и внутренней сетью кластера, что является основой cloud-native архитектуры.